DEADLOCK DETECTION AND RECOVERY SIMULATOR

A Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Gireesh Chowdary — 2520090215

P. Gowtham Varma — 2520090202

A. Siddhartha Varma — 2520090119

Under the guidance of

D Alekya

Assistant Professor

CS&IT

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Year: 2026–2027

DECLARATION

We hereby declare that the project report entitled “Deadlock Detection and Recovery Simulator” is a record of work carried out by us as part of the Operating Systems project, in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering. The work presented in this report is based on the project design and implementation described in the accompanying presentation. Any external references used are acknowledged in the References section.

STUDENT NAME: Gireesh Chowdary ROLL NUMBER: 2520090202

STUDENT NAME: P. Gowtham Varma ROLL NUMBER: 2520090202

STUDENT NAME: A. Siddhartha Varma ROLL NUMBER: 2520090119

Date: _____

Place: _____

CERTIFICATE

This is to certify that the project report entitled “Deadlock Detection and Recovery Simulator” is a bonafide record of the project work carried out by the student(s) named below, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026–2027.

Student name(s): _____

Roll number(s): _____

Signature of the Project Guide: _____

Signature of the Course Coordinator: _____

Signature of the Head of the Department: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and the faculty members of the Department of Computer Science and Engineering for their guidance, encouragement, and valuable suggestions throughout the development of the Deadlock Detection and Recovery Simulator.

We thank the course coordinator and the Head of the Department for providing the academic support and facilities required to complete this Operating Systems project. We also acknowledge the authors of the textbooks and technical documentation consulted while studying deadlock detection, resource allocation, and process recovery.

Finally, we thank our parents, friends, and classmates for their support and cooperation during the completion of this project.

STUDENT NAME(S): _____

ABSTRACT

A deadlock is a condition in an operating system in which a group of processes waits indefinitely for resources held by one another. Such a condition can prevent useful work from progressing and may require intervention to restore the system. The Deadlock Detection and Recovery Simulator is an educational project designed to demonstrate how deadlocked processes can be identified and how a system can recover by terminating selected processes and reclaiming their resources.

The core detection method uses an Allocation matrix, a Request matrix, and an Available vector. A working copy of the available resources is maintained, and the algorithm repeatedly searches for an unfinished process whose outstanding request can be satisfied. When a process is assumed to finish, its allocated resources are returned to the working pool. Processes that remain unfinished when no further progress is possible are reported as deadlocked. The presentation describes a C implementation with limits of 20 processes and 20 resource types, along with a Python extension that demonstrates additional recovery strategies, wait-for graphs, and a live multithreaded example.

The simulator illustrates the four Coffman conditions, completion sequences, detection complexity, and recovery through victim termination. It is intended for learning and demonstration; it does not claim to monitor or repair a production operating system. The project provides a practical way to understand resource allocation, deadlock detection, and recovery in Operating Systems.

INDEX

Chapter	Topics / Subtopics	Page No.
	Acknowledgement	i
	Abstract	ii
1	Introduction	1
1.1	Background of the Project	1
1.2	Problem Statement	1
1.3	Scope of the Project	2
1.4	Objectives of the Project	2
1.5	Importance and Target Users	2
2	OS Concepts and Background Study	3
2.1	Deadlock and Coffman Conditions	3
2.2	Existing Methods / Algorithms	4
2.3	Proposed Approach	4
2.4	Advantages	5
3	System Requirements	6
3.1	Hardware Requirements	6
3.2	Software Requirements	6
3.3	Operating System Environment	6
3.4	Development Tools	6
4	Technology and Tools Used	7
4.1	C Programming and Standard I/O	7
4.2	GCC Compiler	7
4.3	Python Extension	7
4.4	Graphviz and Threading (Optional Extension)	8
5	System Architecture and Design	9
5.1	Overall Architecture	9
5.2	System Workflow	9
5.3	Data Structures	10
5.4	Module Description	10
5.5	Input and Output Design	11
6	Implementation	12

6.1	Input and Initialization	12
6.2	Detection Algorithm	13
6.3	Recovery Algorithm	14
6.4	Pseudocode	14
6.5	Complexity Analysis	15
7	Execution and Demonstration	16
7.1	Compilation and Execution	16
7.2	Sample Input and Output	17
7.3	Recovery Walkthrough	18
7.4	Python Extension Run	18
8	Conclusion	19
8.1	Summary and Objectives Achieved	19
8.2	Limitations	19
8.3	Future Scope	20
9	References	21

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Operating systems coordinate multiple processes that compete for limited resources such as memory, files, devices, and locks. A process may hold one resource while requesting another. If a set of processes waits in a cycle, none of them may be able to proceed. This condition is called deadlock. Understanding deadlock is important because it affects system progress, resource utilization, and application reliability.

The Deadlock Detection and Recovery Simulator models resource allocation and demonstrates a systematic way to detect processes that cannot complete under the current resource state. It then illustrates recovery by terminating a selected process and returning its allocated resources to the available pool.

1.2 Problem Statement

When processes hold resources and request additional resources, circular waiting can prevent all affected processes from progressing. Without a clear detection procedure, it can be difficult for learners to identify which processes are blocked and how releasing resources can restore progress. The project addresses this learning problem by accepting resource-state data, applying a detection algorithm, identifying unfinished processes, and demonstrating recovery.

1.3 Scope of the Project

- Accepts the number of processes and resource types within the limits described in the presentation.
- Uses Allocation and Request matrices together with an Available vector.
- Identifies a possible completion sequence or reports processes left unfinished by detection.
- Demonstrates recovery by terminating a selected simulated process and reclaiming its allocation.
- Describes an extended Python demonstration with wait-for graph visualization and alternative recovery strategies.
- Focuses on simulation and learning; it does not control real operating-system processes.

1.4 Objectives of the Project

- To explain the concept of deadlock and its necessary conditions.
- To implement a resource-based deadlock detection procedure.
- To distinguish a safe completion sequence from a deadlocked state.
- To demonstrate recovery by selecting a victim process and releasing its allocated resources.
- To analyze the algorithm’s time complexity and validate user input.

1.5 Importance of the Application and Target Users

The project is useful for Operating Systems students, instructors, and developers who need a small reproducible model of deadlock behavior. Similar concepts arise in database transaction locks, multithreaded programs, server workloads, and shared-resource systems. The simulator itself is an educational tool, not a production monitoring utility.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Deadlock and Coffman Conditions

A deadlock is a state in which a set of processes is permanently blocked because each process is waiting for a resource or event that can be supplied only by another blocked process. The classical Coffman conditions describe four conditions that must hold together for deadlock to occur:

Condition	Description
Mutual exclusion	At least one resource is non-shareable and can be used by only one process at a time.
Hold and wait	A process holds one or more resources while waiting for additional resources.
No preemption	Resources cannot normally be forcibly removed; they are released voluntarily by the holding process.
Circular wait	A cycle exists in which each process waits for a resource held by the next process in the cycle.

2.2 Existing Methods / Algorithms

Operating systems commonly discuss deadlock prevention, avoidance, detection, and recovery as separate strategies. Prevention attempts to ensure that at least one necessary condition cannot hold. Avoidance makes allocation decisions based on whether the resulting state remains safe, as in the Banker's algorithm. Detection permits resource allocation and periodically checks whether deadlock has occurred. Recovery then restores progress, for example by terminating processes or preempting resources where safe and supported.

The project focuses on detection and recovery. The detection method resembles the standard multiple-resource detection algorithm: it compares each unfinished process's outstanding request with a simulated working vector of available resources.

2.3 Proposed Approach

1. Read the process count, resource-type count, Allocation matrix, Request matrix, and Available vector.
2. Copy Available into a temporary Work vector and initialize Finish flags.
3. Find an unfinished process whose Request row is less than or equal to Work for every resource type.
4. Assume that process completes, add its Allocation row to Work, mark it finished, and record it in the sequence.
5. Repeat until no additional process can be marked finished.
6. Report unfinished processes as deadlocked candidates, then allow recovery by terminating a selected victim and re-running detection.

2.4 Advantages of the Proposed Approach

- Uses clear matrices and vectors that are easy to inspect in a classroom demonstration.
- Produces a completion sequence when processes can finish under the simulated resource state.
- Separates detection from recovery, making each stage easier to understand.
- Validates the documented process and resource limits and rejects invalid victim selections.
- Supports an extended Python demonstration with multiple recovery strategies.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum practical requirement
Processor	Any modern desktop or laptop CPU
Memory	2 GB RAM or more recommended
Storage	A few MB for source files and compiler/runtime
Display	Terminal or command prompt; GUI optional for extended visualizations

3.2 Software Requirements

Software	Purpose
C compiler (GCC or compatible)	Compile the C simulator described in the presentation
Python 3	Run the extended Python simulator, if that source file is available
Terminal / Command Prompt	Enter test data and view detection/recovery results
Graphviz (optional)	Render wait-for or resource-allocation graph exports in the Python extension

3.3 Operating System Environment

The project can be developed and demonstrated on a Linux environment such as Ubuntu, or on Windows using a compatible GCC toolchain or WSL. Exact commands may vary by the installed compiler and the source-file name. The presentation identifies the core C program as using standard input/output and describes the Python extension as runnable with Python 3.

3.4 Development Tools and Compilers

GCC is suitable for compiling a standard C implementation. A text editor or IDE can be used to edit the source code. Python 3 is required only for the extended simulator. Graphviz is optional and is relevant only when graph export is implemented and installed.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C Programming and Standard I/O

The presentation describes the core simulator as a plain C program using stdio.h. C arrays represent the Allocation and Request matrices, Available and Work vectors, Finish flags, and the completion sequence. Standard input and output are sufficient for entering resource data and printing results.

4.2 GCC Compiler

GCC compiles the C source file into an executable. A typical compilation command is shown in Chapter 7; the actual filename should be changed to match the supplied source file.

4.3 Python Extension

The presentation describes a Python version that extends the basic C concept with a wait-for graph, termination of all deadlocked processes, one-by-one victim selection, resource preemption with rollback considerations, and a live multithreaded demonstration. These features should be described as extension capabilities only if they are present in the Python source used for the demonstration.

4.4 Graphviz and Threading

Graphviz can be used to export graph structures as images. Threading can demonstrate real lock interactions and deadlock behavior in a controlled educational program. These are extension topics from the presentation and are not required to compile the basic C implementation.

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The simulator is organized into three stages: Input, Detect, and Recover. Input collects the current resource state. Detection works on a temporary copy of the available resources and identifies a completion order or unfinished processes. Recovery reclaims resources from a chosen victim and invokes detection again.

```
Input data
   |
   v
Validate process/resource counts
   |
   v
Allocation + Request + Available
   |
   v
Deadlock detection (Work / Finish)
   |
   +---- all finished ----> Print completion sequence
   |
   +---- unfinished -----> Report deadlocked processes
                                |
                                v
                          Select victim process
                                |
                                v
                          Reclaim allocation
                                |
                                v
                          Run detection again
```

5.2 System Workflow

7. Start the program and enter valid dimensions for processes and resource types.
8. Enter the Allocation matrix, Request matrix, and Available vector.
9. Initialize Work from Available and set Finish flags based on current allocations.
10. Repeatedly mark processes that can finish and release their simulated allocations into Work.
11. Print a safe completion sequence or identify unfinished processes.
12. If deadlock is reported, select a valid unfinished victim to terminate.
13. Return the victim's allocated resources to Work and repeat detection until the remaining processes can complete.

5.3 Data Structures

Data structure	Dimensions	Purpose
alloc	n × m	Number of instances of each resource currently allocated to each process.
request	n × m	Additional instances of each resource still requested by each process.

avail	m	Currently available instances of each resource type.
work	m	Temporary resource pool used by the detection algorithm.
finish	n	Flag indicating whether a process can be treated as completed or holds no resources.
sequence	n	Records the order in which processes are assumed to complete.

Here, n is the number of processes and m is the number of resource types. The presentation states that the C version supports up to 20 processes and 20 resource types.

5.4 Module Description

Module	**Responsibility**
Input validation	Check dimensions and collect matrix/vector values.
Initialization	Copy Available into Work and initialize Finish.
Detection	Search for processes whose requests can be satisfied; release their allocations in the simulation.
Deadlock reporting	Print the completion sequence or the processes left unfinished.
Recovery	Validate the selected victim, reclaim its allocation, and repeat detection.

5.5 Input and Output Design

Inputs include n, m, an n × m Allocation matrix, an n × m Request matrix, and an m-element Available vector. Outputs include either a completion sequence when no deadlock remains, or a list of unfinished processes identified by detection. During recovery, the simulator prints the victim choice and the processes that become able to complete.

CHAPTER 6: IMPLEMENTATION

6.1 Input and Initialization

The presentation specifies validation of process and resource counts in the range 1 to 20. After reading the matrices and vector, Work is initialized as a copy of Available. A process with no allocated resources is initially marked finished because it cannot be holding resources needed by another process.

```
for (j = 0; j < m; j++)
    work[j] = avail[j];

for (i = 0; i < n; i++) {
    finish[i] = 1;
    for (j = 0; j < m; j++)
        if (alloc[i][j] != 0)
            finish[i] = 0;
}
```

6.2 Detection Algorithm

For each unfinished process i, the algorithm checks whether request[i][j] is less than or equal to work[j] for every resource type j. If so, the process is assumed to complete, and its allocated resources are added back to Work. The process is marked finished and appended to the completion sequence. The algorithm repeats while progress is made. Any process remaining unfinished is reported as deadlocked by the simulator.

```
repeat
    progress = false
    for each process i:  // skip if finish[i] is true
        if request[i][j] <= work[j] for every resource j:
            for each resource j:
                work[j] = work[j] + alloc[i][j]
            finish[i] = true
            append i to sequence
            progress = true
until progress == false

if any finish[i] == false:
    report unfinished processes
else:
    print completion sequence
```

6.3 Recovery Algorithm

Recovery is demonstrated by selecting a process identified as deadlocked. The selection is validated so that invalid or already-finished processes are rejected. The victim is terminated in the simulation, its allocated resources are returned to the working pool, and the detection algorithm is run again. The process repeats until no process remains blocked in the simulated state.

```
while deadlocked processes remain:
    display deadlocked processes
    read victim selection
    if victim is invalid or already finished:
        ask again
    else:
        work = work + alloc[victim]
```

```
        mark victim as terminated/finished
        run detection again
```

6.4 Algorithm / Pseudocode Summary

Stage	Key operation	Result
Input	Read and validate resource state	Consistent matrix/vector dimensions
Detect	Compare Request rows against Work	Completion sequence or unfinished set
Recover	Terminate victim and reclaim Allocation	Updated resource pool
Re-detect	Repeat detection after recovery	Processes that can now finish

6.5 Complexity Analysis

The presentation gives the worst-case detection complexity as $O(m \cdot n^2)$, where n is the number of processes and m is the number of resource types. Each full pass may inspect up to n processes and m resource types, and at most n successful completions are possible before a pass makes no progress. The C implementation described in the slides limits both n and m to 20, keeping the classroom demonstration small.

6.6 Robustness and Edge Cases

- Reject process or resource counts outside the supported range.
- Treat processes holding no resources as finished during initialization.
- Reject invalid victim numbers and selections that refer to already-finished processes.
- Stop detection when a complete pass makes no progress.
- Do not confuse a safe completion order with a recovery sequence; recovery is invoked only when unfinished processes are reported.

CHAPTER 7: EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu / Linux

Save the C source file from the project as deadlock_detector.c (or use the actual filename). Compile with GCC:

```
gcc deadlock_detector.c -o deadlock_detector -Wall -Wextra
```

Run the program using:

```
./deadlock_detector
```

On Windows, use a GCC-compatible environment such as MinGW or WSL and run the generated executable according to that environment. The command above assumes the source code contains a main function and uses standard C.

7.2 Python Extension Run

If the Python extension file described in the presentation is included, run it using Python 3:

```
python3 deadlock_simulator.py
```

The presentation identifies the Python extension as supporting wait-for graph display, terminate-all recovery, one-by-one victim selection, preemption/rollback concepts, live threads, and optional Graphviz export. Availability of each feature depends on the actual source file and installed dependencies.

7.3 Sample Resource State

Process	Allocation (A B C)	Request (A B C)
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 1
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

Available = 0 0 0. The presentation lists the total resources as A = 7, B = 2, and C = 6. In the safe version, the displayed completion sequence is P0, P2, P3, P4, P1. The example then changes P2's request so that it asks for one additional C instance; the presentation reports P1, P2, P3, and P4 as deadlocked, while P0 can finish.

7.4 Recovery Walkthrough

Step	Action	Work vector after action
0	P0 completes and releases (0,1,0)	0 1 0
1	Terminate P1; reclaim (2,0,0)	2 1 0
2	P3 completes; reclaim (2,1,1)	4 2 1
3	P2 completes; reclaim (3,0,3)	7 2 4
4	P4 completes; reclaim (0,0,2)	7 2 6

The presentation describes the recovery concept as one victim termination followed by re-detection and completion of the remaining processes. The vector sequence above illustrates resource reclamation; exact printed sequence depends on the implemented program and input.

7.5 Screenshots of Execution

Insert screenshots from your own terminal after compiling and running the actual source code. Recommended screenshots are: (1) successful compilation, (2) safe input and completion sequence, (3) modified request showing deadlocked processes, and (4) victim selection followed by successful recovery. Do not insert fabricated output; use screenshots generated by the project.

7.6 Demonstration of OS Concepts

- Resource allocation is represented using matrices and vectors.
- A completion sequence shows how processes may finish as resources are released.
- Deadlock detection identifies processes that cannot finish under the current simulated state.
- Recovery demonstrates termination and resource reclamation.

CHAPTER 8: CONCLUSION

8.1 Summary of the Project

The Deadlock Detection and Recovery Simulator provides a structured demonstration of deadlock in an operating system. It uses Allocation and Request matrices and an Available vector to simulate the resource state. Its detection procedure repeatedly finds processes whose requests can be met, returns their allocations to the temporary resource pool, and identifies processes that remain unfinished. Recovery is demonstrated by terminating a selected victim and repeating detection.

8.2 Objectives Achieved

- Explained the four Coffman conditions and their relationship to deadlock.
- Presented the detection algorithm and the role of Work and Finish.
- Demonstrated completion sequences and deadlock reporting using a sample resource state.
- Explained recovery by victim termination and resource reclamation.
- Documented compilation, execution, complexity, and validation considerations.

8.3 Limitations

The C simulator is described as a small educational program with a maximum of 20 processes and 20 resource types. The algorithm operates on user-provided resource-state data rather than inspecting a live operating system. The report is based on the supplied presentation; actual screenshots, exact source code listings, and the precise runtime output should be added from the final project files before submission.

8.4 Future Scope

- Automatically select a victim using priority, process age, or termination cost.
- Add deadlock avoidance using the Banker’s algorithm.
- Explore resource preemption and checkpoint/rollback where safe and supported.
- Create a graphical interface that animates resource allocation and wait-for graphs.
- Expand test cases and add automated validation for invalid inputs and edge conditions.

CHAPTER 9: REFERENCES

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts, Wiley.

2. Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, Pearson.

3. GNU Compiler Collection (GCC), official documentation: https://gcc.gnu.org/onlinedocs/

4. Python 3 Documentation: https://docs.python.org/3/

5. Project presentation supplied by the student: “Deadlock Detection and Recovery Simulator.”

Note: Verify the exact edition/year of textbooks and include any additional sources actually consulted.